ELEC 328 Project Report
Assn 1- Procedural Mesh Interactions
John Tucker – S1439352 – jrt15
ELEC 328 Project Report
Assn 2 – Enchanted VR Flashlight
John Tucker – S1439352 – jrt15

1. Concept

[Screenshot 1]

The goal of this assignment was to learn how materials, shaders, and lighting can change the look and feel of a space without changing the actual geometry. We had to build an enchanted flashlight in Shader Graph that shows a normal texture everywhere except inside the flashlight's cone where a hidden texture is reveale and use a C# script to send the flashlight's position, direction, and angle to the shader to tell it when to reveal. For my project I built a foggy, rainy Victorian style back alley at night kinda inspired by like Jack the Ripper meets scooby doo, laid out along the Roost Rails. The cobblestones and brick look ordinary until you shine the flashlight on them which reveals a trail of bloody footprints leading to a dead end where a spooky sigil is drawn on the wall in blood next to an ominous dagger. .

2. Technical overview

Enchanted flashlight shader

The main system is a URP Lit Shader Graph that every enchanted surface in the scene uses. Each material has its own _BaseMap, _HiddenTexture, _BaseTiling, _BoundaryColor, _BoundaryWidth, and _EdgeFeather. The flashlight values (_Flashlight_Origin, _Flashlight_Direction, _Flashlight_ConeAngle) are set to Global and hidden from the Inspector, so that itts easy to update every material at once instead of setting each material separately. The math is in a Custom Function node EnchantedFlashlight.hlsl that runs for every fragment and outputs two masks between 0 and 1 named Reveal and Boundary.

Inside the cone (Objective C): for each pixel the function takes its world-space position P and finds the unit vector from the flashlight to the surface, L = normalize(P − origin). The dot product of L with the beam direction D is the cosine of the angle between the center of the beam and pixel. I turn it back into an angle with acos and subtract it from the cone half-angle, so edgeDist is positive inside the cone and negative outside. Working in degrees made the boundary width and feather easier to tune in the Inspector.

Facing away (Objective D): a surface should only be revealed if it faces the flashlight, so the function takes the dot product of the surface normal N with −L. If it is negative the surface is facing away and step() makes the mask 0, so the default texture stays. Using step and smoothstep instead of if statements keeps the shader branch-free, which is better for the GPU like we mentioned in class.

Boundary (Objective E): To get the outline around the beam, I basically took two cones and subtract one from the other. The first one, inCone, is the full flashlight cone. The second, inCore, is slightly smaller, shrunk by _BoundaryWidth degrees. Whatever’s left over between them is a thin ring right at the edge of the beam, and that’s the boundary. In code that’s just Reveal = inCore × facing and Boundary = (inCone − inCore) × facing, and both use smoothstep so

_EdgeFeather can soften the edges and keep them from looking weird. Back in the Shader Graph, I sample the base and hidden textures, blend between them using Reveal, then blend toward _BoundaryColor using Boundary before plugging it all into Base Color. Since it's a Lit graph, the ring and the hidden texture still react to the real lights in the scene instead of looking pasted on

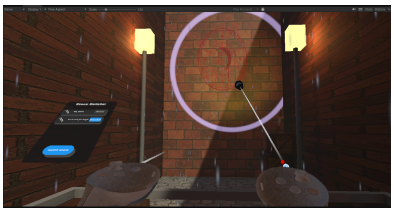
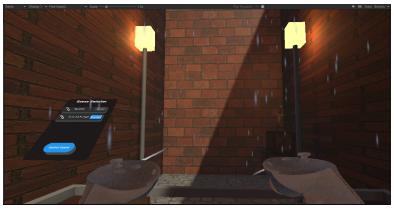
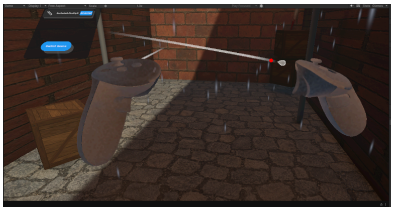
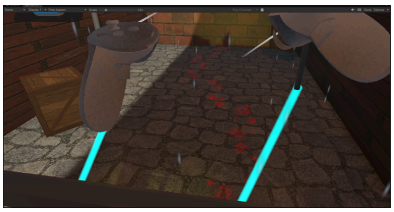
Flashlight object and script (FlashlightController.cs, Objective A)
The flashlight is a grabbable object made with a Rigidbody and an XR Grab Interactable. BeamOrigin, sits at the lens with its forward axis pointing out of the lens and shoots out a Spot Light. FlashlightController sends BeamOrigin's world position, forward direction, and cone angle to the shader with Shader.SetGlobalVector and Shader.SetGlobalFloat. It also sets the Spot Light's spot angle to twice the cone half-angle so the real light matches the shader's cone. Pulling the trigger or clicking Toggles, which sets the cone angle to 0 so nothing is revealed and turns the light off. I added a GripPoint and turned off dynamic attach and throw on detach so the flashlight always sits in the hand pointing forward because it was behaving weird.

Materials and textures (Objectives B and F)
The alley floor is a large cobblestone plane using the provided texture with high smoothness so it looks wet. In front of the rails, is a smaller path of cobble made of that uses M_Floor_Footprints which is a cobblestone base with a bloody footprint trail as the hidden texture. The shader only tiles the base map, so stretching one plane made the footprints huge. Instead I used a footprint texture whose ends line up and placed copies end to end, with Base Tiling (1.25, 2.5) so the stones match the floor around it. The side walls use WornBrick. The end wall uses M_Wall_Sigil, with a brick base and the blood sigil as the hidden texture. The sigil texture has transparent padding around the symbol instead of covering the whole wall. Each enchanted material has a deep red boundary color for spooky effect.

Lighting and atmosphere (theme)
For the night mood the ambient light is a dark blue the directional light is a dim blue moonlight, the skybox is a starry space skybox, and I added fog that fades the end of the alley. The main lights are gas lamps and window lights which are warm orange point lights with GasLampFlicker.cs and just a normal point light for the windows . Each lamp head also has an XR Simple Interactable and LampToggle.cs, so the player can click a lamp to turn its light on and off. Rain is a Particle System with a box emitter over the alley. Crates and a dagger near the sigil wrap the scene up nicely and give it the vibe I want .

Reality Roost: the alley is built around the Roost Rails prefab, and the RR_Boot scene provides the VR rig for my example video but I remove the before building with the RR-Host profile (RR_Boot, RR_Calib, Enchanted_Flashlight).

3. Evidence of requirements

Requirement	Where it is demonstrated	Screenshot
Flashlight Sigil	Both are demonstrated heavily in the first minute of the video	 
Flashlight Steps		 

5.)
The only asset I used that wasn't generated by claude or given in the textures folder was the skybox which I found for the last assignment here:
https://assetstore.unity.com/packages/2d/textures-materials/sky/free-skyboxes-space-178953
by Dogmatic

6.)

Tool	Used for	What was incorporated, changed, or rejected
Claude	Generating Textures/Materials	Claude helped me with generating the textures for the sigil and the bloody footsteps. As well as the dagger.
Claude	Debugging Enchanted Flashlight	Claude helped me go from the rough math/code I did and the ideas to a real shader graph
Claude	Extra scripts	Wrote GasLampFlicker.cs and LampToggle.cs. I built the lamps and set up the interactables and events.
Claude	Unity guidance and debugging	Walked me through the floor and wall tiling, rain, flashlight grip, and lamps.
Claude	General troubleshooting	I ran into a ton of issues not knowing where to find certain tabs or why my code was acting weird and Claude helped a lot.

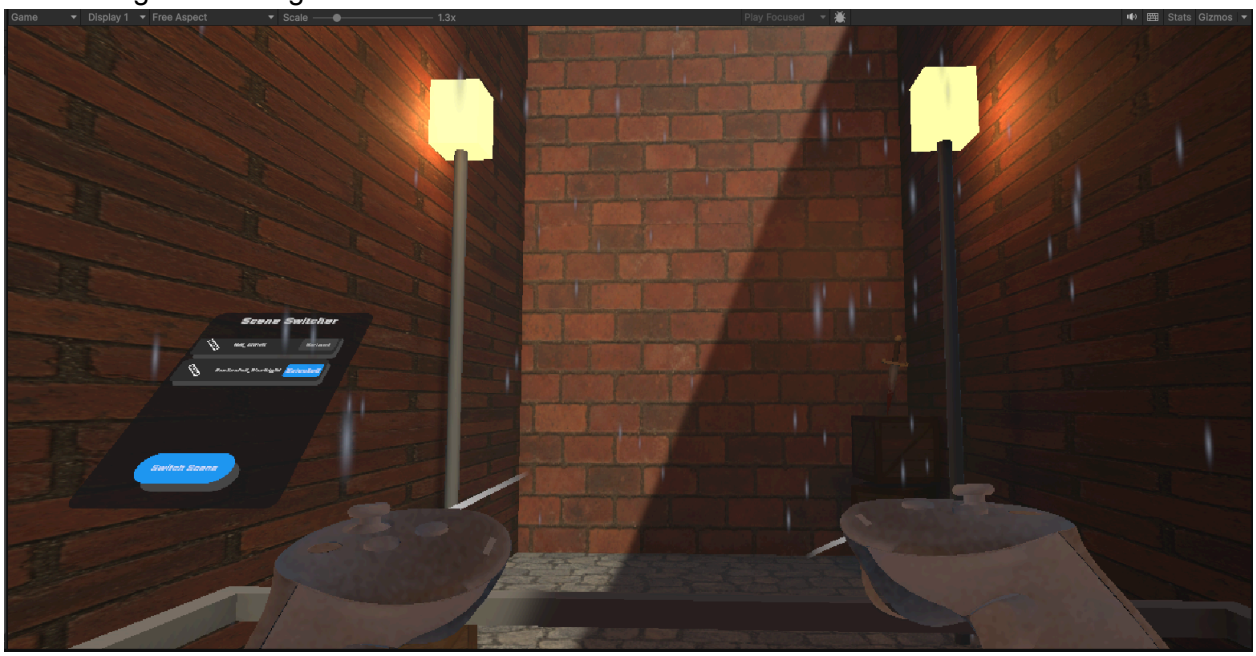
7.)
What worked best was the shader itself. Once the dot product idea clicked, the reveal, the facing check, and the boundary ring all came pretty easily. It was cool to come up with cool ideas for the flashlight. The hardest part for me was Unity itself rather than the math, since I'm new to the editor. Textures stretched when I scaled a plane until I learned to use tiling, the footprints came out huge, the flashlight sat in my hand at weird angles until I added an attach point, and my rain was spawning but not moving because the velocity was 0. I also lost time to setup problems and a corrupted save that deleted my rain so I had to rebuild it. If I had more time I'd animate the hidden layer with scrolling noise so the blood looks wet, wiggle vertices inside the beam, and replace the lamps and props with real models. The biggest thing I learned is how much lighting, fog, and materials change the mood because I was worried about how rough it was looking at first.

One of the big issues I ran into is that I could get the flashlight and lamp clicking to work with the roost origins on my desktop but not when I actually went to the roost. The experience video shows the general experience and vibe and you can still see the flashlight shining on the floor and revealing the hidden footprints. I think I messed up in getting the interactables to work like I did in assignment 1 but it is shown heavily in my desktop video.

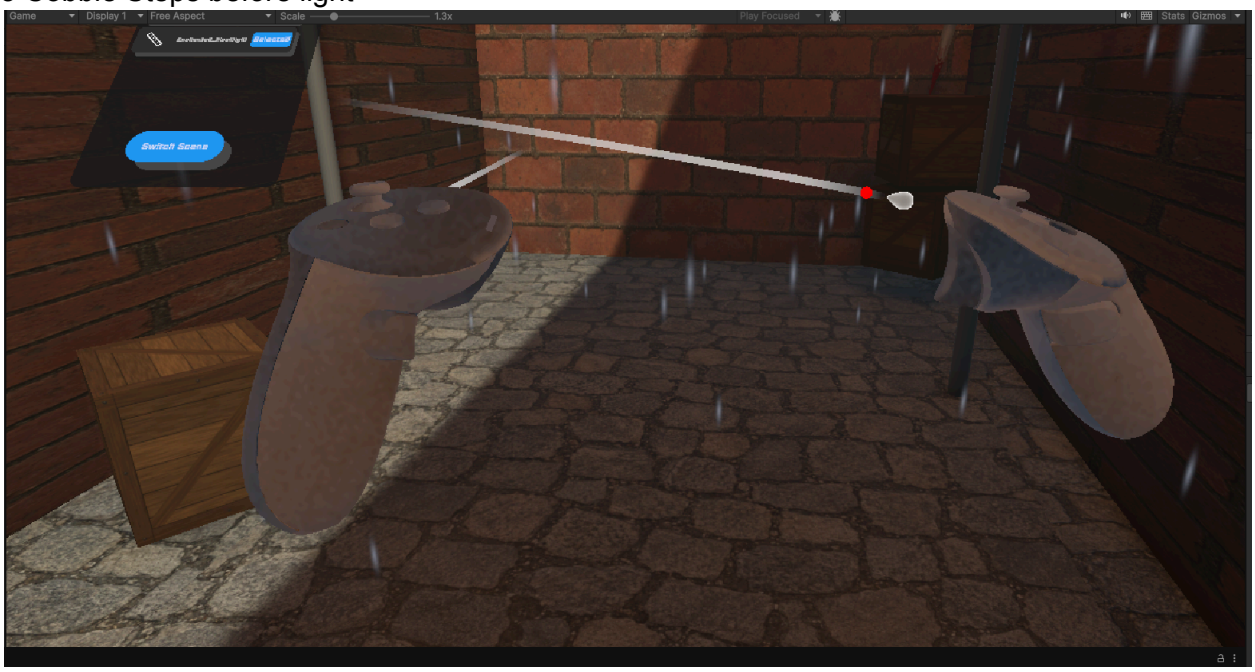
Screenshots: Screenshots
1 Blood Sigil After Light



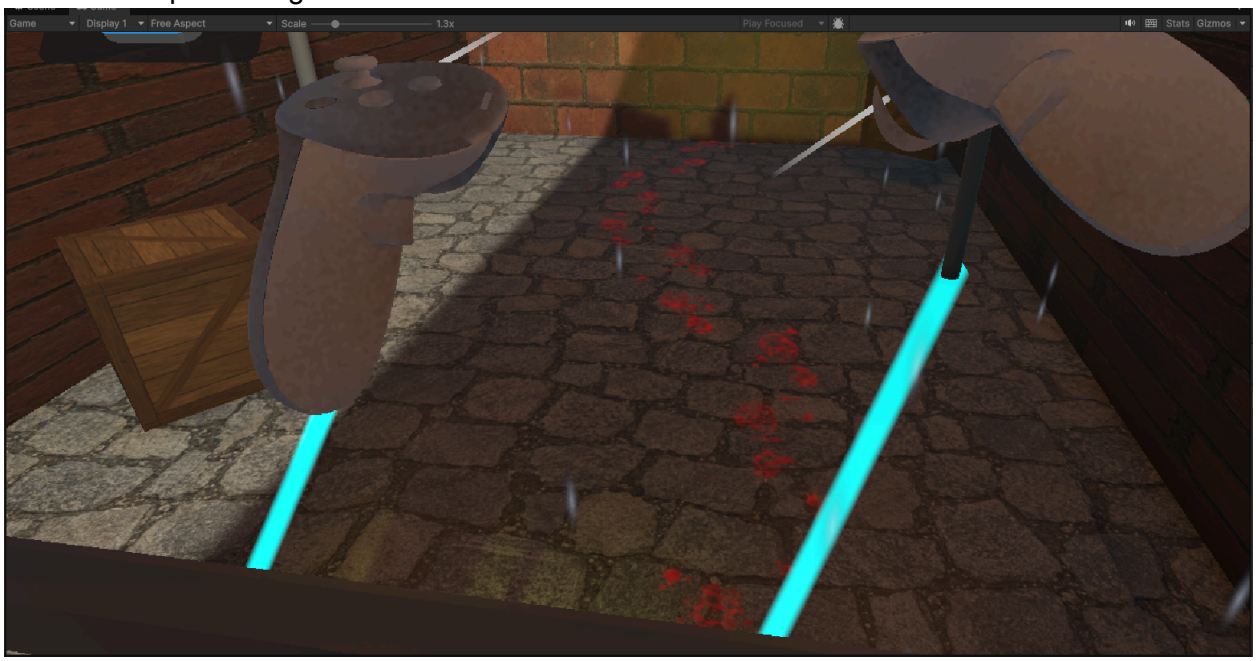
2 Blood Sigil Before Light



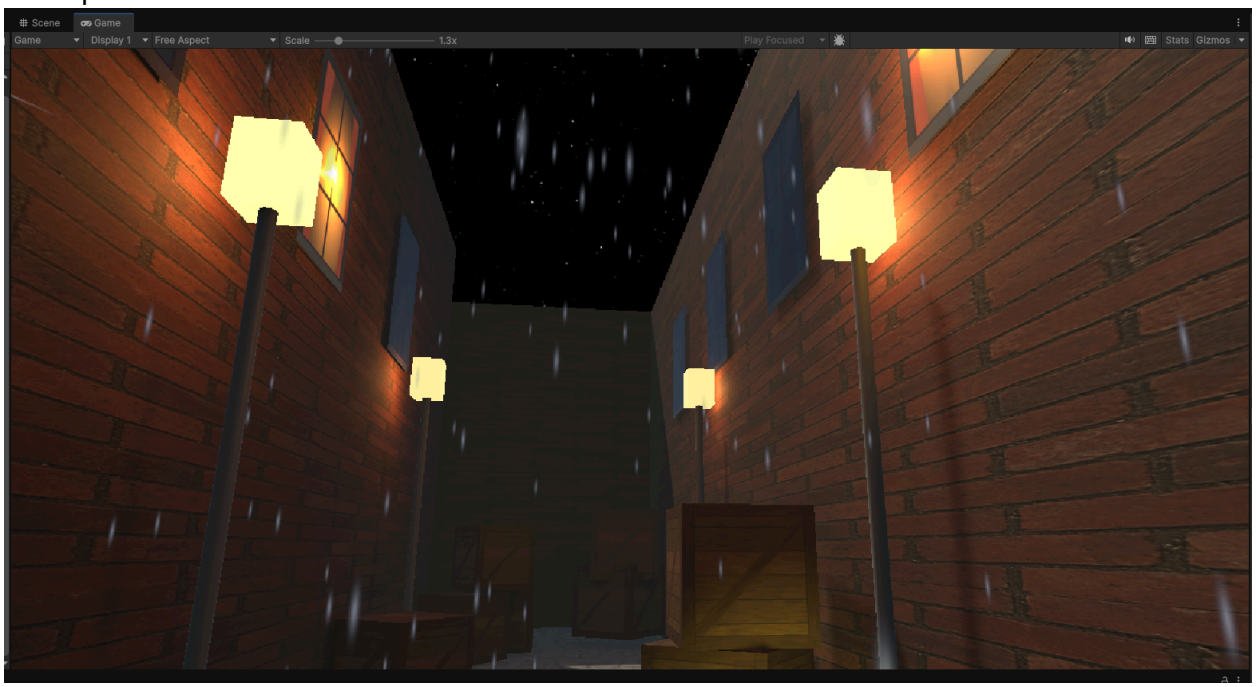
3 Cobble Steps before light



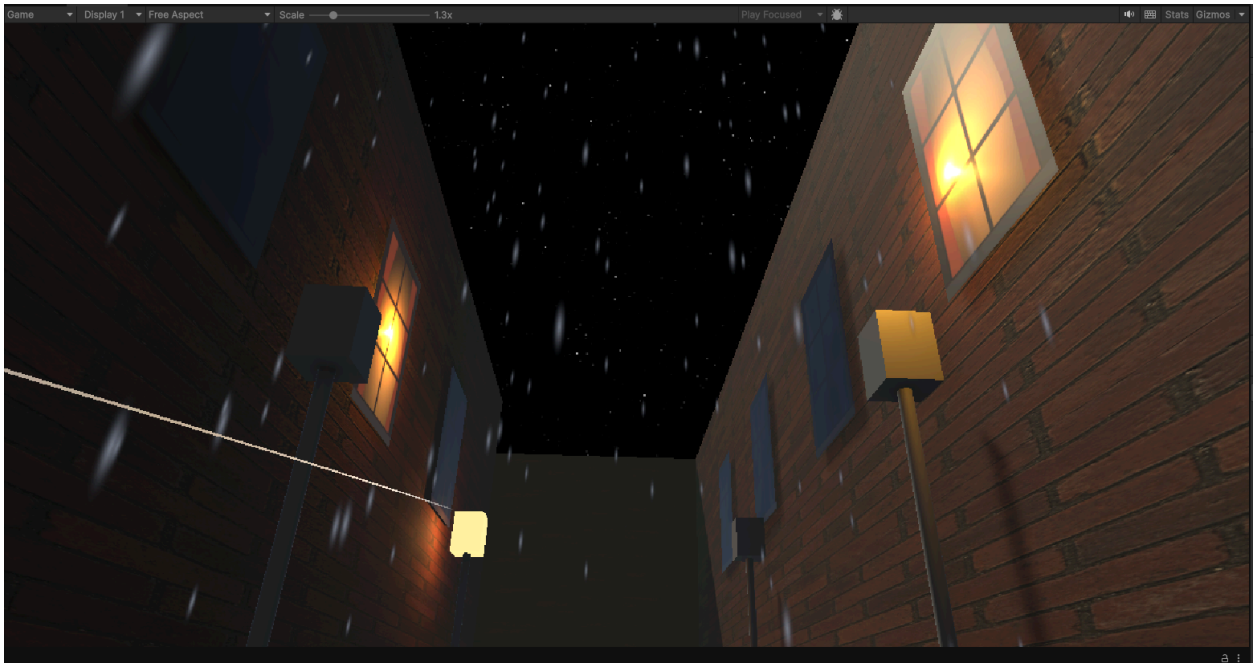
4 Cobble Steps after light



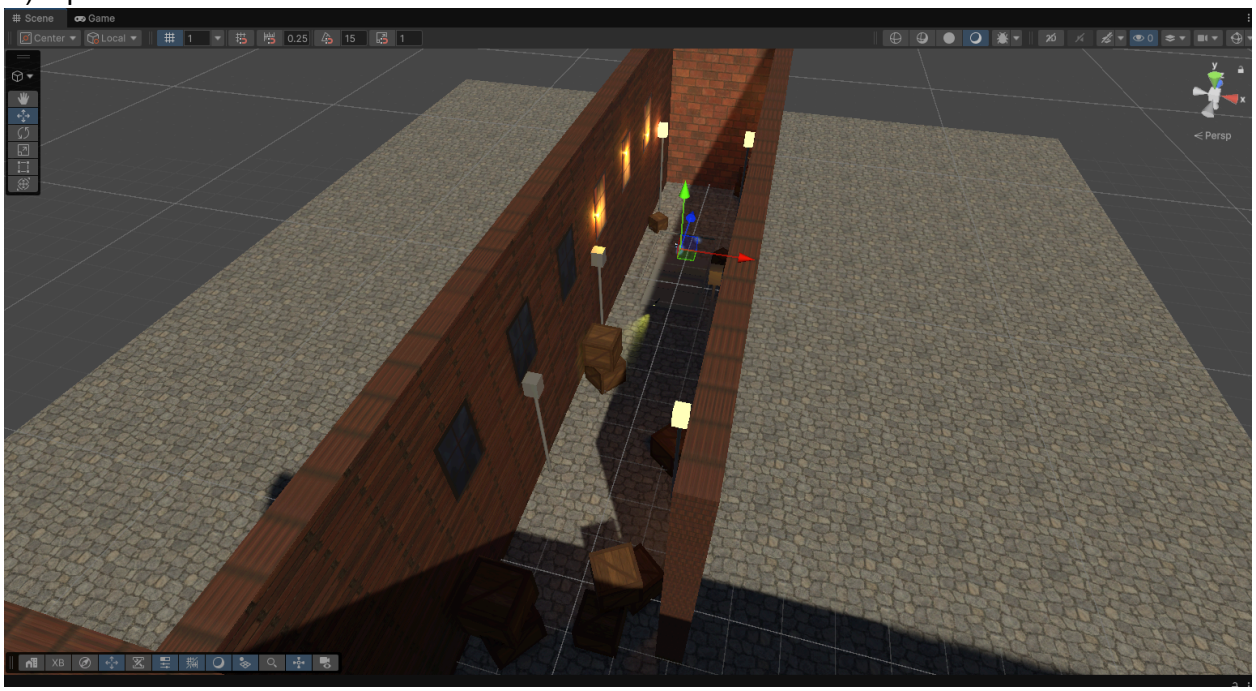
5 Lamps and windows



6 Lamps turned off and better window view



7.) top down view



8.) Spooky Dagger

